

NNSDK C/C++ API Guide

Version: v0.1

Release Date: 2026-05-31

版权

© 2026 晶晨半导体（上海）股份有限公司（代表其自身及其关联公司，以下简称“Amlogic”）版权所有。未经 Amlogic 书面许可，不得以任何形式或通过任何手段对本文件的任何部分进行复制、传播、转录或翻译成任何语言。

商标

 Amlogic，及 Amlogic 的其他图标均为 Amlogic 公司的商标。所有其他商标和注册商标均为其各自所有人的财产。

免责声明

由于产品版本升级或其他原因，本文档将可能在未经任何通知的情况下，不定期进行更新或修改。

本产品不适用于医疗、救生或生命维持等用途，也不适用于可合理预见因 Amlogic 产品故障或失灵可能导致人身伤害、死亡、严重财产或环境损害的应用场景。

文中提供的电路图及其他与 Amlogic 产品相关的信息仅为典型应用说明，完整且足以用于生产设计的全部信息不一定会提供。Amlogic 已尽量确保本文件内容准确和及时，但 Amlogic 不对本文档所含信息的准确性或完整性作任何保证。

Amlogic 产品可能存在未被识别的漏洞，或可能支持具有已知局限性的既定安全标准或规范。Amlogic 对任何漏洞不承担任何责任。

使用相关材料可能需获得第三方或 Amlogic 的知识产权许可。本文档对归属于 Amlogic 或任何其他方的知识产权不授予任何明示或暗示的声明或保证。文中包含的任何第三方网站链接仅为方便用户而提供，Amlogic 不对该等网站及其任何行为（包括隐私惯例、可用性和内容）作出认可，也不承担任何责任。

在任何情况下，Amlogic 均不对与本文档相关、由本文档使用或无法使用所产生的任何索赔承担责任。

联系方式

- 公司网址: www.amlogic.cn
- 售前咨询: contact@amlogic.com
- 技术支持: support@amlogic.com

修订记录

版本	修改说明
v0.1 (2026-05-31)	初始版本

目录

1. C API 说明

2 基础数据结构定义

- amlnn_query_cmd
- amlnn_sdk_version
- amlnn_input_output_num
- amlnn_tensor_format
- amlnn_tensor_type
- amlnn_tensor_attr
- amlnn_input_range
- amlnn_profiling_data
- amlnn_perf_run
- amlnn_npu_core_status
- amlnn_input
- amlnn_output
- amlnn_backend_type
- amlnn_model_task_priority
- amlnn_init_config
- amlnn_run_config
- amlnn_dma_memory
- amlnn_ext_mem_access_mode
- amlnn_ext_buffer_type
- amlnn_ext_io_type
- amlnn_dma_config
- amlnn_softop_type
- amlnn_softop_acc_type
- amlnn_softop_opt_request
- amlnn_core_mask_policy
- amlnn_core_mask_config

3 基础API介绍

- amlnn_init
- amlnn_query
- amlnn_inputs_set
- amlnn_run
- amlnn_outputs_get
- amlnn_destroy
- amlnn_alloc_dma_mem
- amlnn_bind_dma_mem
- amlnn_unbind_dma_mem
- amlnn_free_dma_mem
- amlnn_set_softop_opt
- amlnn_set_input_shapes
- amlnn_set_core_mask

4 错误返回码

1. C API 说明

C API 目前已升级至第二代，数据结构及接口见于 `nnsdk2.h`，如客户想了解或者继续使用第一代的 C API，可参考 NNSDK1 的文档说明

2 基础数据结构定义

本章节按 `nnsdk2.h` 中的顺序列出所有数据结构。

amlnn_query_cmd

枚举类型，用于 `amlnn_query` 的命令选择。

成员	说明
AMLNN_QUERY_SDK_VERSION	查询 SDK 与驱动版本
AMLNN_QUERY_IN_OUT_NUM	查询输入输出数量
AMLNN_QUERY_INPUT_ATTR	查询输入张量属性
AMLNN_QUERY_OUTPUT_ATTR	查询输出张量属性
AMLNN_QUERY_INPUT_DYNAMIC_RANGE	查询输入张量的动态 shape 范围
AMLNN_QUERY_CURRENT_INPUT_ATTR	查询当前输入 shape（仅动态模型有效）
AMLNN_QUERY_CURRENT_OUTPUT_ATTR	查询当前输出 shape（仅动态模型有效）
AMLNN_QUERY_PERF_DETAIL	查询 profiling 数据并触发性能详情文本文件输出； 需在 <code>amlnn_init</code> 时，启用 <code>amlnn_init_config</code> 结构体中的 <code>enable_perf</code> ， 并设置 <code>perf_detail_path</code> ，且需在 <code>amlnn_run</code> 之后调用；仅支持 ADLA NPU
AMLNN_QUERY_PERF_RUN	查询 run 的耗时；需在 <code>amlnn_run</code> 之后调用
AMLNN_QUERY_MODEL_SOFTOP	查询模型的软件算子信息（TODO）
AMLNN_QUERY_NPU_CORE_STATUS	查询 npu core 的状态

amlnn_sdk_version

结构体 `amlnn_sdk_version` 表示 SDK 版本信息，其定义如下：

成员变量	数据类型	含义
<code>sdk2_api_version</code>	<code>char[]</code>	aml nnsdk2 api 版本
<code>adla_kmd_version</code>	<code>char[]</code>	ADLA 驱动版本（NPU）
<code>delegate_version</code>	<code>char[]</code>	delegate 版本（GPU/CPU）

amlnn_input_output_num

结构体 amlnn_input_output_num 表示模型输入输出的数量，其定义如下：

成员变量	数据类型	含义
n_input	uint32_t	输入数量
n_output	uint32_t	输出数量

amlnn_tensor_format

枚举类型，表示张量数据布局格式。

成员	说明
AMLNN_TENSOR_NHWC	NHWC 格式
AMLNN_TENSOR_NCHW	NCHW 格式

amlnn_tensor_type

枚举类型，表示张量数据类型。

成员	说明
AMLNN_TENSOR_FLOAT32	FP32 类型
AMLNN_TENSOR_FLOAT16	FP16 类型
AMLNN_TENSOR_INT8	INT8 类型
AMLNN_TENSOR_UINT8	UINT8 类型
AMLNN_TENSOR_INT16	INT16 类型
AMLNN_TENSOR_UINT16	UINT16 类型
AMLNN_TENSOR_INT32	INT32 类型
AMLNN_TENSOR_UINT32	UINT32 类型
AMLNN_TENSOR_INT64	INT64 类型
AMLNN_TENSOR_UINT64	UINT64 类型
AMLNN_TENSOR_BOOL	BOOL 类型
AMLNN_TENSOR_INT4	INT4 类型
AMLNN_TENSOR_BFLOAT16	BF16 类型

amlnn_tensor_attr

结构体 amlnn_tensor_attr 表示模型输入/输出张量的属性，其定义如下：

成员变量	数据类型	含义
index	uint32_t	输入参数：输入/输出张量索引
n_dims	uint32_t	维度数
dims	uint32_t[]	维度数组
name	char[]	张量名称
n_elems	uint32_t	元素数量
size	uint32_t	张量字节数
fmt	amlnn_tensor_format	张量数据布局格式
type	amlnn_tensor_type	张量数据类型
zp	int32_t	零点参数
scale	float	scale 参数

amlnn_input_range

结构体 amlnn_input_range 表示一个输入的支持形状列表信息。包含了输入的索引、支持的形状个数、数据布局格式、名称以及形状列表，其定义如下：

成员变量	数据类型	含义
index	uint32_t	输入参数：输入张量索引
shape_number	uint32_t	动态 shape 数量
fmt	amlnn_tensor_format	张量数据布局格式
name	char[]	张量名称
dyn_range	uint32_t[] []	动态输入维度范围
n_dims	uint32_t	维度数

amlnn_profiling_data

结构体 amlnn_profiling_data 表示性能 profiling 数据（仅支持 ADLA NPU），其定义如下：

成员变量	数据类型	含义
us_elapsed_in_hw_op	uint64_t	硬件算子执行耗时
us_elapsed_in_sw_op	uint64_t	软件算子执行耗时
memory_usage_bytes	uint64_t	内存占用

成员变量	数据类型	含义
dram_read_bytes	uint64_t	DDR 读流量（带宽）
dram_write_bytes	uint64_t	DDR 写流量（带宽）
sram_read_bytes	uint64_t	SRAM 读流量（带宽）
sram_write_bytes	uint64_t	SRAM 写流量（带宽）
axi_freq_cur	uint64_t	当前 NPU AXI 频率
core_freq_cur	uint64_t	当前 NPU core 频率
mem_allocated_base	uint64_t	驱动基础占用内存
mem_allocated_umd	uint64_t	当前模型占用内存
mem_pool_size	uint64_t	驱动支持的地址空间大小
mem_pool_used	uint64_t	驱动已占用总内存（mem_allocated_base + mem_allocated_umd(modelA) + ...）
reserved	uint8_t[]	预留

amlInn_perf_run

结构体 amlInn_perf_run 表示单次 run 耗时统计，其定义如下：

成员变量	数据类型	含义
run_duration	int64_t	实际推理耗时（us）

amlInn_npu_core_status

结构体 amlInn_npu_core_status 表示 npu core 的状态（仅支持 ADLA NPU），其定义如下：

成员变量	数据类型	含义
core_num	uint32_t	当前设备 npu core 的数量
core_mask	uint64_t	当前设备 npu core 的掩码，以 bit 位呈现
idle_mask	uint64_t	当前处于空闲状态的 npu core 掩码
busy_mask	uint64_t	当前处于忙碌状态的 npu core 掩码
reserved	uint8_t[]	预留

amlInn_input

结构体 amlInn_input 表示模型的一个数据输入，用来作为参数传入给 amlInn_inputs_set 函数，其定义如下：

成员变量	数据类型	含义
------	------	----

成员变量	数据类型	含义
index	uint32_t	输入索引
buf	void*	输入数据缓冲区指针
size	uint32_t	输入数据缓冲区字节数

amlnn_output

结构体 amlnn_output 表示模型的一个数据输出，用来作为参数传入给 amlnn_outputs_get 函数，在函数执行后，结构体对象将会被赋值，其定义如下：

成员变量	数据类型	含义
is_float	uint8_t	是否需要 float 类型的输出
index	uint32_t	输出索引
buf	void*	输出数据缓冲区指针
size	uint32_t	输出数据缓冲区字节数

amlnn_backend_type

枚举类型，用于指定后端类型。

成员	说明
AMLNN_BACKEND_ADLA_NPU	运行 ADLA 模型（ADLA NPU）
AMLNN_BACKEND_TF_DELEGATE_GPU	运行 TFLite 模型（GPU）
AMLNN_BACKEND_TF_DELEGATE_CPU	运行 TFLite 模型（CPU）

amlnn_model_task_priority

枚举类型，用于指定模型任务优先级。

成员	说明
AMLNN_MODEL_TASK_PRIOR_MEDIUM	模型任务为中优先级
AMLNN_MODEL_TASK_PRIOR_LOW	模型任务低优先级
AMLNN_MODEL_TASK_PRIOR_HIGH	模型任务高优先级

amlnn_init_config

结构体 amlnn_init_config 表示 amlnn_init 初始化配置（仅支持 ADLA NPU），其定义如下：

成员变量	数据类型	含义
backend_type	amlnn_backend_type	后端类型

成员变量	数据类型	含义
flags	uint32_t	初始化标志位 (TODO)
task_priority	amlInn_model_task_priority	模型任务优先级
timeout_ms	uint32_t	invoke 超时时间
enable_perf	uint32_t	是否获取模型性能数据
perf_detail_path	const char*	性能数据文本输出路径
reserved	uint8_t[]	预留

amlInn_run_config

结构体 amlInn_run_config 表示 amlInn_run 的运行配置，其定义如下：

成员变量	数据类型	含义
reserved	uint8_t[]	预留

amlInn_dma_memory

结构体 amlInn_dma_memory 表示 DMA 内存信息（仅支持 ADLA NPU），其定义如下：

成员变量	数据类型	含义
handle	void*	内部内存句柄
viraddr	void*	虚拟地址
phyaddr	void*	物理地址
index	uint32_t	内存索引标识
size	uint32_t	申请的内存大小

amlInn_ext_mem_access_mode

枚举类型，表示 DMA 访问模式（仅支持 ADLA NPU）。

成员	说明
AMLNN_EXT_MEM_ACCESS_READ_WRITE	CPU 可读可写
AMLNN_EXT_MEM_ACCESS_READ_ONLY	CPU 只读 (TODO)
AMLNN_EXT_MEM_ACCESS_WRITE_ONLY	CPU 只写 (TODO)

amlnn_ext_buffer_type

枚举类型，表示 DMA 地址类型（仅支持 ADLA NPU）。

成员	说明
AMLNN_EXT_BUFFER_VIRTUAL_ADDR	虚拟地址
AMLNN_EXT_BUFFER_PHYS_ADDR	物理地址
AMLNN_EXT_BUFFER_SECURE_PHYS_ADDR	secure 物理地址

amlnn_ext_io_type

枚举类型，表示 DMA 绑定方向（仅支持 ADLA NPU）。

成员	说明
AMLNN_EXT_IO_INPUT	绑定为输入
AMLNN_EXT_IO_OUTPUT	绑定为输出

amlnn_dma_config

结构体 amlnn_dma_config 表示 DMA 配置（仅支持 ADLA NPU），其定义如下：

成员变量	数据类型	含义
cacheable	uint32_t	启用缓存以加快 CPU 访问速度
access_mode	amlnn_ext_mem_access_mode	CPU 访问类型
buffer_type	amlnn_ext_buffer_type	buffer 地址类型
io_type	amlnn_ext_io_type	输入或输出

amlnn_softop_type

枚举类型，表示软件算子类型（仅支持 ADLA NPU）。该枚举较长，下表仅列出部分示例，完整列表见 `nnsdk2.h`。

成员	说明
AMLNN_Add	Add 类型
AMLNN_Conv2d	Conv2d 类型
AMLNN_DepthwiseConv2d	DepthwiseConv2d 类型
AMLNN_Relu	Relu 类型
AMLNN_Softmax	Softmax 类型
AMLNN_Unknown	Unknown 类型

amlnn_softop_acc_type

枚举类型，表示软件算子加速类型（仅支持 ADLA NPU）。

成员	说明
AMLNN_SOFTOP_ACC_OPENMP	OpenMP 加速
AMLNN_SOFTOP_ACC_VULKAN	Vulkan 加速（TODO）
AMLNN_SOFTOP_ACC_OPENCL	OpenCL 加速（TODO）

amlnn_softop_opt_request

结构体 amlnn_softop_opt_request 表示软件算子加速配置（仅支持 ADLA NPU），其定义如下：

成员变量	数据类型	含义
softop_type	amlnn_softop_type	软件算子类型
acc_type	amlnn_softop_acc_type	软件算子加速类型

amlnn_core_mask_policy

枚举类型，表示设置 npu core 掩码的策略（仅支持 ADLA NPU）。

成员	说明
AMLNN_CORE_MASK_AUTO	由系统自动调度选择执行任务的 npu core
AMLNN_CORE_MASK_BEST_EFFORT	尝试使用用户请求的 npu core，但不保证一定能用上

amlnn_core_mask_config

结构体 amlnn_core_mask_config 表示 npu core 掩码配置信息（仅支持 ADLA NPU），其定义如下：

成员变量	数据类型	含义
requested_mask	uint64_t	请求的 npu core 掩码
policy	amlnn_core_mask_policy	选择的 npu core 掩码策略
reserved	uint8_t[]	预留

3 基础API介绍

amlnn_init

amlnn_init 初始化函数功能为创建 amlnn context 对象、根据所选后端类型选择加载 ADLA / TFLite 模型以及根据 amlnn_init_config 结构体执行特定的初始化行为。

API	amlnn_init
功能	初始化 amlnn 上下文句柄
参数	void** context: 上下文句柄双指针
	void* model: 模型的二进制数据或者模型路径（若所选后端非 ADLA NPU, 则只支持传模型路径） 当参数 size 大于0时, model 表示二进制数据 当参数 size 等于0时, model 表示模型路径
	uint32_t size: 当 model 是二进制数据, 表示模型大小, 当 model 是路径, 则设置为 0
	amlnn_init_config* init_config: 初始化配置参数
返回值	int 错误码（见 AMLNN API 错误返回码 ）

示例代码如下：

```
void *ctx = NULL;
amlnn_init_config init_config;
memset(&init_config, 0, sizeof(amlnn_init_config));
ret = amlnn_init(&ctx, (void *)model_path, 0, &init_config);
```

amlnn_query

amlnn_query 函数能够查询获取到：SDK 版本、模型输入输出信息、profiling 数据、逐层性能详情文本文件、模型推理的总时间等信息。

API	amlnn_query
功能	查询模型与 SDK 的相关信息
参数	void* context: 上下文句柄指针
	amlnn_query_cmd : 查询命令
	void* info: 存放返回结果的结构体变量
	uint32_t size: info 对应的结构体变量的大小
返回值	int 错误码（见 AMLNN API 错误返回码 ）

当前 SDK 支持的查询命令如下表所示：

查询命令	返回结果结构体	功能
AMLNN_QUERY_SDK_VERSION	amlnn_sdk_version	查询 SDK 版本
AMLNN_QUERY_IN_OUT_NUM	amlnn_input_output_num	查询输入输出张量个数
AMLNN_QUERY_INPUT_ATTR	amlnn_tensor_attr	查询输入张量属性
AMLNN_QUERY_OUTPUT_ATTR	amlnn_tensor_attr	查询输出张量属性
AMLNN_QUERY_INPUT_DYNAMIC_RANGE	amlnn_input_range	使用支持动态形状模型时， 查询模型支持输入形状数量、 列表、形状对应的数据布局 和名称等信息
AMLNN_QUERY_CURRENT_INPUT_ATTR	amlnn_tensor_attr	使用支持动态形状模型时， 查询模型当前推理所使用的输入 属性
AMLNN_QUERY_CURRENT_OUTPUT_ATTR	amlnn_tensor_attr	使用支持动态形状模型时， 查询模型当前推理所使用的输出 属性
AMLNN_QUERY_PERF_DETAIL	amlnn_profiling_data	查询 profiling 数据并导出逐 层性能详情文本文件， 需要在调用 amlnn_init 接口 时， 启用 amlnn_init_config 结构 体中的 enable_perf， 并设置 perf_detail_path， 才能生效
AMLNN_QUERY_PERF_RUN	amlnn_perf_run	查询推理模型（不包含设置输入/输出）的耗时， 单位是微秒

各个指令用法的详细说明如下：

1. 查询 SDK 版本

传入 AMLNN_QUERY_SDK_VERSION 命令可以查询 SDK 的版本信息。其中需要先创建 amlnn_sdk_version 结构体对象。示例代码如下：

```
amlnn_sdk_version sdk_ver;
ret = amlnn_query(NULL, AMLNN_QUERY_SDK_VERSION, &sdk_ver, sizeof(sdk_ver));
printf("verion info: sdk2-[%s], adla-[%s], delegate-[%s]\n",
sdk_ver.sdk2_api_version, sdk_ver.adla_kmd_version,
sdk_ver.delegate_version);
```

2. 查询输入输出张量个数

在 amlnn_init 接口调用完毕后，传入 AMLNN_QUERY_IN_OUT_NUM 命令可以查询模型输入输出张量的个数。其中需要先创建 amlnn_input_output_num 结构体对象。示例代码如下：

```
amlnn_input_output_num io_num;
ret = amlnn_query(ctx, AMLNN_QUERY_IN_OUT_NUM, &io_num, sizeof(io_num));
printf("model input num: %d, output num: %d\n", io_num.n_input,
io_num.n_output);
```

3. 查询输入张量属性

在 `amlnn_init` 接口调用完毕后，传入 `AMLNN_QUERY_INPUT_ATTR` 命令可以查询模型输入张量的属性。其中需要先创建 `amlnn_tensor_attr` 结构体对象。示例代码如下：

```
amlnn_tensor_attr input_attrs[io_num.n_input];
memset(input_attrs, 0, sizeof(input_attrs));
for (uint32_t i = 0; i < io_num.n_input; i++)
{
    input_attrs[i].index = i;
    ret = amlnn_query(ctx, AMLNN_QUERY_INPUT_ATTR, &(input_attrs[i]),
        sizeof(amlnn_tensor_attr));
}
```

4. 查询输出张量属性

在 `amlnn_init` 接口调用完毕后，传入 `AMLNN_QUERY_OUTPUT_ATTR` 命令可以查询模型输出张量的属性。其中需要先创建 `amlnn_tensor_attr` 结构体对象。示例代码如下：

```
amlnn_tensor_attr output_attrs[io_num.n_output];
memset(output_attrs, 0, sizeof(output_attrs));
for (uint32_t i = 0; i < io_num.n_output; i++)
{
    output_attrs[i].index = i;
    ret = amlnn_query(ctx, AMLNN_QUERY_OUTPUT_ATTR, &(output_attrs[i]),
        sizeof(amlnn_tensor_attr));
}
```

5. 查询模型支持的动态输入形状信息

在 `amlnn_init` 接口调用完毕后，传入 `AMLNN_QUERY_INPUT_DYNAMIC_RANGE` 命令可以查询模型支持的输入形状信息，包含输入形状个数、输入形状列表、输入形状对应的布局和名称等信息。其中需要先创建 `amlnn_input_range` 结构体对象。示例代码如下：

```
amlnn_input_range shape_range[io_num.n_input];
memset(shape_range, 0, io_num.n_input * sizeof(amlnn_input_range));
for (uint32_t i = 0; i < io_num.n_input; i++)
{
    shape_range[i].index = i;
    ret = amlnn_query(ctx, AMLNN_QUERY_INPUT_DYNAMIC_RANGE, &shape_range[i],
        sizeof(amlnn_input_range));
}
```

6. 查询模型当前使用的输入动态形状

在 `amlnn_set_input_shapes` 接口调用完毕后，传入 `AMLNN_QUERY_CURRENT_INPUT_ATTR` 命令可以查询模型当前使用的输入属性信息。其中需要先创建 `amlnn_tensor_attr` 结构体对象。示例代码如下：

```

amlnn_tensor_attr cur_input_attrs[io_num.n_input];
memset(cur_input_attrs, 0, io_num.n_input * sizeof(amlnn_tensor_attr));
for (uint32_t i = 0; i < io_num.n_input; i++)
{
    cur_input_attrs[i].index = i;
    ret = amlnn_query(ctx, AMLNN_QUERY_CURRENT_INPUT_ATTR, &
(cur_input_attrs[i]), sizeof(amlnn_tensor_attr));
}

```

7. 查询模型当前使用的输出动态形状

在 amlnn_set_input_shapes 接口调用完毕后，传入 AMLNN_QUERY_CURRENT_OUTPUT_ATTR 命令可以查询模型当前使用的输出属性信息。其中需要先创建 amlnn_tensor_attr 结构体。示例代码如下：

```

amlnn_tensor_attr cur_output_attrs[io_num.n_output];
memset(cur_output_attrs, 0, io_num.n_output * sizeof(amlnn_tensor_attr));
for (uint32_t i = 0; i < io_num.n_output; i++)
{
    cur_output_attrs[i].index = i;
    ret = amlnn_query(ctx, AMLNN_QUERY_CURRENT_OUTPUT_ATTR, &
(cur_output_attrs[i]), sizeof(amlnn_tensor_attr));
}

```

8. 查询 profiling 数据并导出逐层性能详情文本文件

在 amlnn_run 接口调用完毕后，amlnn_query 接口传入 AMLNN_QUERY_PERF_DETAIL 可以获取 profiling 数据并导出逐层性能详情文本文件。使用该命令的前提是，在 amlnn_init 接口的 amlnn_init_config 结构体中，启用 enable_perf，并设置 perf_detail_path 作为文本文件的输出路径。示例代码如下：

```

void *ctx = NULL;
amlnn_init_config init_config;
memset(&init_config, 0, sizeof(amlnn_init_config));
init_config.enable_perf = 1;
init_config.perf_detail_path = "/data/nn";
ret = amlnn_init(&ctx, (void *)model, model_len, &init_config);
...
ret = amlnn_run(ctx, NULL);
...
amlnn_profiling_data profiling_data;
memset(&profiling_data, 0, sizeof(amlnn_profiling_data));
ret = amlnn_query(ctx, AMLNN_QUERY_PERF_DETAIL, &profiling_data,
sizeof(profiling_data));
// 调用完毕后，会在 “/data/nn” 路径下生成 profile_info.txt 文件，记录了该模型本次推理
的逐层性能详情，包括耗时、带宽等。

```

9. 查询模型推理的总耗时

在 amlnn_run 接口调用完毕后，amlnn_query 接口传入 AMLNN_QUERY_PERF_RUN 可以查询模型推理（不包含设置输入/输出）的耗时，单位是微秒。示例代码如下：


```
amlnn_perf_run perf_run;
memset(&perf_run, 0, sizeof(amlnn_perf_run));
ret = amlnn_query(ctx, AMLNN_QUERY_PERF_RUN, &perf_run, sizeof(perf_run));
```

amlnn_inputs_set

通过 amlnn_inputs_set 函数可以设置模型的输入数据。该函数能够支持多个输入，其中每个输入是 amlnn_input 结构体对象，在传入之前用户需要设置该对象。

API	amlnn_inputs_set
功能	设置模型输入数据
参数	void* context: 上下文句柄指针
	uint32_t n_inputs: 输入数据个数
	amlnn_input inputs[]: 输入数据数组，数组每个元素是 amlnn_input 结构体对象
返回值	int 错误码（见 AMLNN API 错误返回码 ）

示例代码如下：

```
amlnn_input inputs[1];
memset(inputs, 0, sizeof(inputs));
inputs[0].index = 0;
inputs[0].buf = input_data; // 需要在外层做好前处理，包括归一化、量化、数据类型匹配，数据布局格式匹配等，sdk 内部会将该块数据直接拷贝到 npu buf
inputs[0].size = input_size; // 输入数据字节数
ret = amlnn_inputs_set(ctx, io_num.n_input, inputs);
```

amlnn_run

amlnn_run 函数将执行一次模型推理，调用之前需要先通过 amlnn_inputs_set 函数或者 DMA 接口设置输入数据。

API	amlnn_run
功能	执行一次模型推理
参数	void* context: 上下文句柄指针
	amlnn_run_config* run_config: 推理配置参数，当前未使用，传入 NULL 即可
返回值	int 错误码（见 AMLNN API 错误返回码 ）

示例代码如下：

```
ret = amlnn_run(ctx, NULL);
```

amlnn_outputs_get

amlnn_outputs_get 函数可以获取模型推理的输出数据。该函数能够一次获取多个输出数据。其中每个输出是 amlnn_output 结构体对象，在函数调用之前需要依次创建并设置每个 amlnn_output 结构体对象。

API	amlnn_outputs_get
功能	获取模型推理输出
参数	void* context: 上下文句柄指针
	uint32_t n_outputs: 输出数据个数
	amlnn_output outputs[]: 输出数据的数组，其中数组每个元素为 amlnn_output 结构体对象，代表模型的一个输出
返回值	int 错误码（见 AMLNN API 错误返回码 ）

示例代码如下：

```
amlnn_output outputs[1];
memset(outputs, 0, sizeof(outputs));
outputs[0].is_float = 1;
outputs[0].index = 0;
ret = amlnn_outputs_get(ctx, 1, outputs);
```

amlnn_destroy

amlnn_destroy 函数将释放传入的 amlnn context 及其相关资源。

API	amlnn_destroy
功能	销毁 amlnn context 对象及其相关资源
参数	void* context: 上下文句柄指针
返回值	int 错误码（见 AMLNN API 错误返回码 ）

```
ret = amlnn_destroy(ctx);
```

amlnn_alloc_dma_mem

仅供 ADLA NPU 使用。

当用户要 NPU 内部分配 DMA 内存时，amlnn_alloc_dma_mem 函数可以分配用户指定的内存大小及类型，类型包括 cache 类型、access 类型、buffer 类型、io 类型，内存大小通过 amlnn_dma_memory 结构体对象来指定，内存类型通过 amlnn_dma_config 结构体对象来指定，在函数调用之前需要创建并设置 amlnn_dma_memory 和 amlnn_dma_config 结构体对象。

DMA 内存分配注意事项：

- **虚拟地址模式**：必须调用 `amlnn_alloc_dma_mem` 函数进行分配，目前**不支持**用户私自映射或分配虚拟地址。
- **物理地址模式**：无需调用 `amlnn_alloc_dma_mem` 函数，由用户自行提供，但需确保物理首地址满足 **4K** 对齐要求。

DMA 内存空间对齐要求：

为兼容硬件的访存步长要求，在使用 DMA 内存时需区分“分配空间”与“使用空间”：

- **分配准则**：申请 DMA 内存的实际大小必须以 `amlnn_tensor_attr` 结构体中的 **size** 为准，若分配不足 `size` 大小，可能会导致硬件越界访问，从而引发系统崩溃或数据异常。
- **使用准则**：在逻辑处理或数据填充时，有效数据范围应以 `amlnn_tensor_attr` 结构体中的 **size** 为准。

API	amlnn_alloc_dma_mem
功能	分配 DMA 内存
参数	<code>void* context</code> ：上下文句柄指针
	<code>amlnn_dma_memory* dma_memory</code> ：DMA 内存信息结构体对象，用于描述一个 DMA buf
	<code>amlnn_dma_config* dma_config</code> ：DMA 内存配置参数
返回值	<code>int</code> 错误码（见 AMLNN API 错误返回码 ）

示例代码如下：

```
amlnn_dma_config dma_config_input;
memset(&dma_config_input, 0, sizeof(amlnn_dma_config));
dma_config_input.cacheable = 1;
dma_config_input.access_mode = AMLNN_EXT_MEM_ACCESS_READ_WRITE;
dma_config_input.buffer_type = AMLNN_EXT_BUFFER_VIRTUAL_ADDR;
dma_config_input.io_type = AMLNN_EXT_IO_INPUT;

amlnn_dma_memory dma_memory_input[1];
memset(dma_memory_input, 0, sizeof(dma_memory_input));
dma_memory_input[0].index = 0;
dma_memory_input[0].size = input_attrs[0].size;
ret = amlnn_alloc_dma_mem(ctx, &dma_memory_input[0], &dma_config_input);
```

```

amlnn_dma_config dma_config_output;
memset(&dma_config_output, 0, sizeof(amlnn_dma_config));
dma_config_output.cacheable = 1;
dma_config_output.access_mode = AMLNN_EXT_MEM_ACCESS_READ_WRITE;
dma_config_output.buffer_type = AMLNN_EXT_BUFFER_VIRTUAL_ADDR;
dma_config_output.io_type = AMLNN_EXT_IO_OUTPUT;

amlnn_dma_memory dma_memory_output[1];
memset(dma_memory_output, 0, sizeof(dma_memory_output));
dma_memory_output[0].index = 0;
dma_memory_output[0].size = output_attrs[0].size;
ret = amlnn_alloc_dma_mem(ctx, &dma_memory_output[0], &dma_config_output);

```

amlnn_bind_dma_mem

仅供 ADLA NPU 使用。

分配完 DMA 内存后，在调用 amlnn_run 前，通过 amlnn_bind_dma_mem 函数可以使该 DMA 内存与 NPU 进行绑定。

API	amlnn_bind_dma_mem
功能	绑定并让 NPU 使用 DMA 内存
参数	void* context: 上下文句柄指针
	amlnn_dma_memory* dma_memory: DMA 内存信息结构体对象，用于描述一个 DMA buf
	amlnn_dma_config* dma_config: DMA 内存配置参数
返回值	int 错误码（见 AMLNN API 错误返回码 ）

示例代码如下：

```

ret = amlnn_bind_dma_mem(ctx, &dma_memory_input[0], &dma_config_input);
ret = amlnn_bind_dma_mem(ctx, &dma_memory_output[0], &dma_config_output);

```

amlnn_unbind_dma_mem

仅供 ADLA NPU 使用。

使用完 DMA 内存后，通过 amlnn_unbind_dma_mem 函数可以将所有之前跟 NPU 绑定的 DMA 内存进行解绑。

API	amlnn_unbind_dma_mem
功能	解绑 DMA 内存
参数	void* context: 上下文句柄指针
返回值	int 错误码（见 AMLNN API 错误返回码 ）

示例代码如下：

```
ret = amlnn_unbind_dma_mem(ctx);
```

amlnn_free_dma_mem

仅供 ADLA NPU 使用。

amlnn_free_dma_mem 函数会释放 amlnn_alloc_dma_mem 函数创建出来的 DMA 内存。

API	amlnn_free_dma_mem
功能	释放 DMA 内存
参数	void* context: 上下文句柄指针
	amlnn_dma_memory* dma_memory: DMA 内存信息结构体对象，用于描述一个 DMA buf
	amlnn_dma_config* dma_config: DMA 内存配置参数
返回值	int 错误码（见 AMLNN API 错误返回码 ）

示例代码如下：

```
ret = amlnn_free_dma_mem(ctx, &dma_memory_input[0], &dma_config_input);  
ret = amlnn_free_dma_mem(ctx, &dma_memory_output[0], &dma_config_output);
```

amlnn_set_softop_opt

仅供 ADLA NPU 使用。

amlnn_set_softop_opt 函数可以配置软件算子的加速选项。

API	amlnn_set_softop_opt
功能	配置软件算子加速选项
参数	void* context: 上下文句柄指针
	amlnn_softop_opt_request requests: 软件算子加速配置数组，其中数组每个元素为 amlnn_softop_opt_request 结构体对象
	uint32_t request_num: 配置数组个数
返回值	int 错误码（见 AMLNN API 错误返回码 ）

示例代码如下：

```
amlnn_softop_opt_request softop_opt_requests[] =
{
    {
        .softop_type = AMLNN_Unknown,           // 需要设置的算子类型
        .acc_type = AMLNN_SOFTOP_ACC_OPENMP,    // 选择 OpenMP 加速
    },
};
ret = amlnn_set_softop_opt(ctx, softop_opt_requests, sizeof(softop_opt_requests)
/ sizeof(amlnn_softop_opt_request));
```

amlnn_set_input_shapes

对于动态形状输入模型，在推理前必须指定当前使用的输入形状。该接口传入输入个数和 amlnn_tensor_attr 数组，包含了每个输入形状和对应的数据布局信息，每个 amlnn_tensor_attr 结构体对象的索引、名称、形状（dims）和内存布局信息（fmt）必须填充，amlnn_tensor_attr 结构体其他成员无需设置。在使用 amlnn_set_input_shapes 函数前，可先通过 amlnn_query 函数查询模型支持的输入形状数量和动态形状列表，要求输入数据的形状在模型支持的输入形状列表中。初次运行或每次切换新的输入形状，需要调用该接口设置新的形状，否则，不需要重复调用该接口。

API	amlnn_set_input_shapes
功能	设置模型当前使用的输入形状
参数	void* context：上下文句柄指针
	uint32_t n_inputs：输入张量的数量
	amlnn_tensor_attr *：输入张量的属性数组指针，传递所有输入的形状信息，用户需要设置 每个输入属性结构体中的index、name、dims、fmt、n_dims成员，其他成员无需设置
返回值	int 错误码（见 AMLNN API 错误返回码 ）

示例代码如下：

```
for (uint32_t i = 0; i < io_num.n_input; i++)
{
    for (uint32_t j = 0; j < input_attrs[i].n_dims; ++j)
    {
        input_attrs[i].dims[j] = shape_range[i].dyn_range[s][j];
    }
}

ret = amlnn_set_input_shapes(ctx, io_num.n_input, input_attrs);
```

amlnn_set_core_mask

仅供 ADLA NPU 使用。

amlnn_set_core_mask 函数用来指定工作的 npu core。

API	amlnn_set_core_mask
-----	---------------------

API	amlnn_set_core_mask
功能	指定工作的 npu core
参数	void* context: 上下文句柄指针
	amlnn_core_mask_config* core_mask_config: npu core 掩码配置信息
返回值	int 错误码 (见 AMLNN API 错误返回码)

示例代码如下：

```
amlnn_core_mask_config core_mask_config;
memset(&core_mask_config, 0, sizeof(amlnn_core_mask_config));
core_mask_config.requested_mask = 0x1; // 使用第一个 npu core
core_mask_config.policy = AMLNN_CORE_MASK_BEST_EFFORT;
ret = amlnn_set_core_mask(ctx, &core_mask_config);
```

4 错误返回码

AMLNN API 函数的返回值错误码定义如下表所示：

返回值	说明
AMLNN_SUCCESS (0)	执行成功
AMLNN_ERR_FAIL (-1)	执行失败（通用错误）
AMLNN_ERR_TIMEOUT (-2)	执行超时
AMLNN_ERR_DEVICE_UNAVAILABLE (-3)	设备不可用
AMLNN_ERR_MALLOC_FAIL (-4)	内存分配失败
AMLNN_ERR_PARAM_INVALID (-5)	参数非法
AMLNN_ERR_MODEL_INVALID (-6)	模型非法
AMLNN_ERR_CTX_INVALID (-7)	上下文非法
AMLNN_ERR_INPUT_INVALID (-8)	输入非法
AMLNN_ERR_OUTPUT_INVALID (-9)	输出非法
AMLNN_ERR_NOT_SUPPORTED (-10)	功能不支持
AMLNN_ERR_NOT_READY (-11)	结果未就绪
AMLNN_ERR_NOT_MATCH (-12)	不匹配